



Retrieval-Augmented Generation

COMP4551

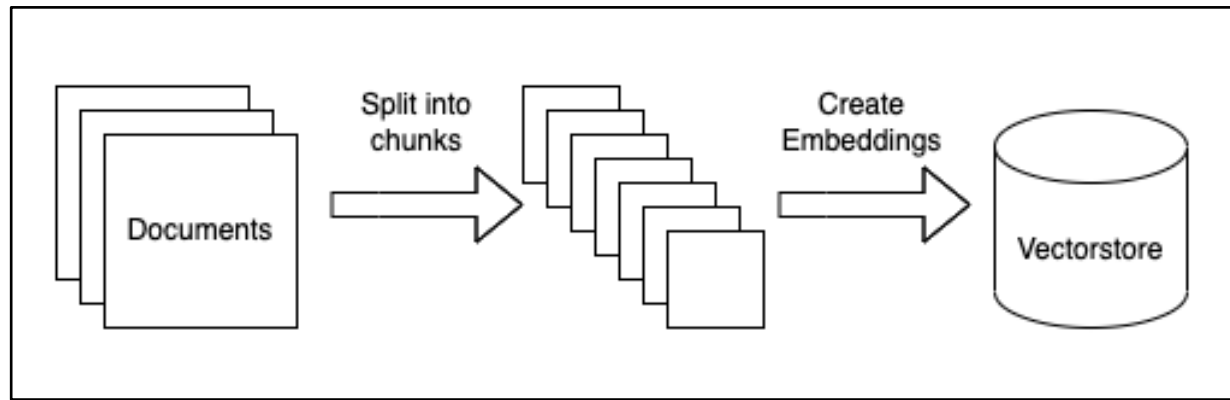
Binhang Yuan

Standard RAG

Retrieval Augmented Generation

- *Retrieval Augmented Generation (RAG)* leverages both generative models and retrieval models for knowledge-intensive tasks.
- It improves Generative AI applications by providing up-to-date information and domain-specific data from external data sources during response generation, reducing the risk of *hallucinations* and significantly improving performance and accuracy.
 - LLM hallucinations: LLMs produce outputs that are coherent and grammatically correct but factually incorrect or nonsensical.
- Two components to set up RAG:
 - Indexing: ingestion of the data.
 - Retrieval and generation: query the data and augment the generation process with additional information.

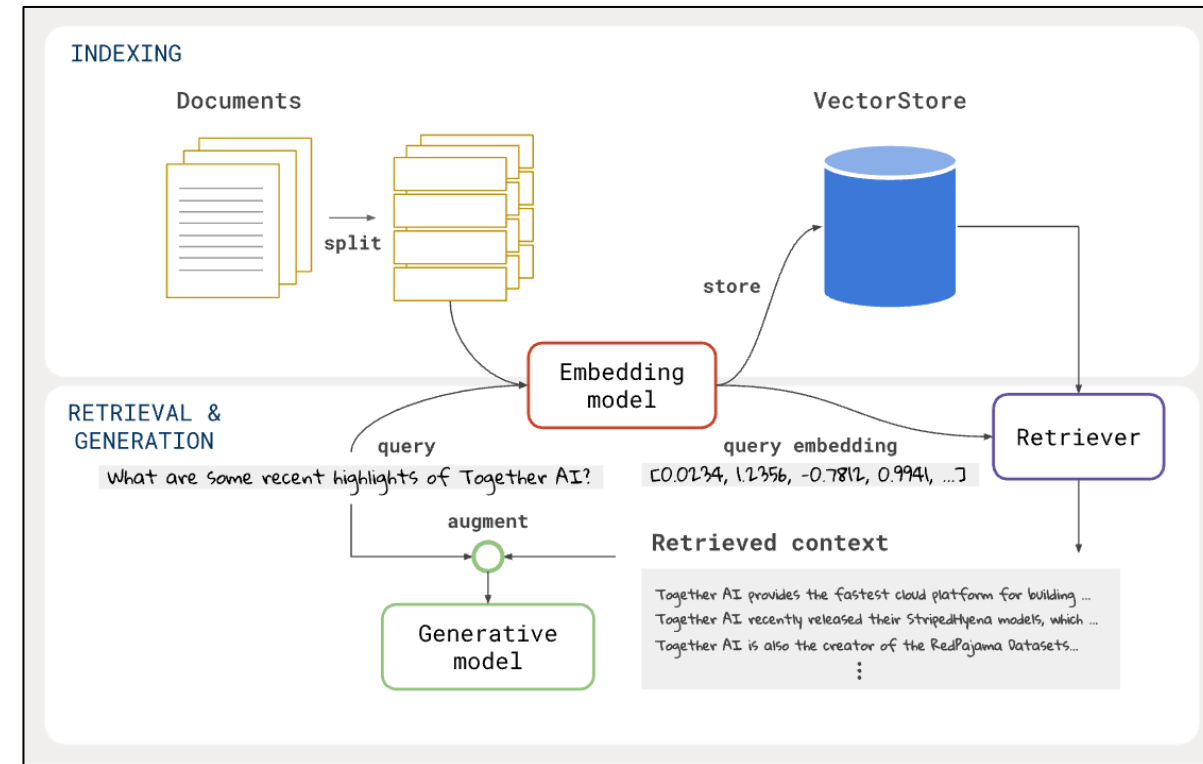
RAG Indexing



- **Load data sources to text**: load data from arbitrary sources to text in a form that can be used downstream.
- **Chunk text**: chunk the loaded text into smaller chunks. This is necessary because LLMs generally have a limit to a context length limitation, so creating as small chunks of text as possible is necessary.
- **Embed text**: create a numerical embedding for each chunk of text. This is necessary because we only want to select the most relevant chunks of text for a given question, and we will do this by finding the most similar chunks in the embedding space.
- **Load embeddings to vectorDB**: put embeddings and documents into a vectorDB. which helps us find the most similar chunks in the embedding space quickly and efficiently.

RAG Retrieval and Generation

- **Generate the embedding of the input prompt**: this is usually to compute the contextual embedding by the embedding model.
- **Lookup relevant documents**: using the embedding of the input prompt to search the vectorDB created during the indexing phase.
- **Augment the prompt**: Combine the new generation input with the retrieved text into a single prompt.
- **Generate a response**: Given the augmented question, we can use an LLM to generate a response.



- LangChain (<https://github.com/langchain-ai/langchain>): A popular open-source framework that provides building blocks for LLM applications.
- Offer easy integration of LLMs with retrieval components.
- LangChain has ready-made classes for splitting text, creating embeddings, storing vectors, and a RetrievalQA chain that ties a retriever and an LLM together for question-answering.
- Developers can use LangChain to construct custom RAG pipelines without reinventing common steps.

RAG Tools LangChain Example

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

# 1. Embed and index documents (offline step)
documents = [...] # list of your text documents
embed_model = OpenAIEmbeddings() # embedding model (e.g., OpenAI API)
vector_store = FAISS.from_texts(documents, embedding=embed_model)
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# 2. Set up the RAG chain with an LLM and the retriever
llm = OpenAI(model_name="text-davinci-003", temperature=0)
rag_chain = RetrievalQA(llm=llm, retriever=retriever)

# 3. Ask a question (runtime step)
query = "What is the annual leave policy for employees?"
response = rag_chain.run(query)
print(response)
```

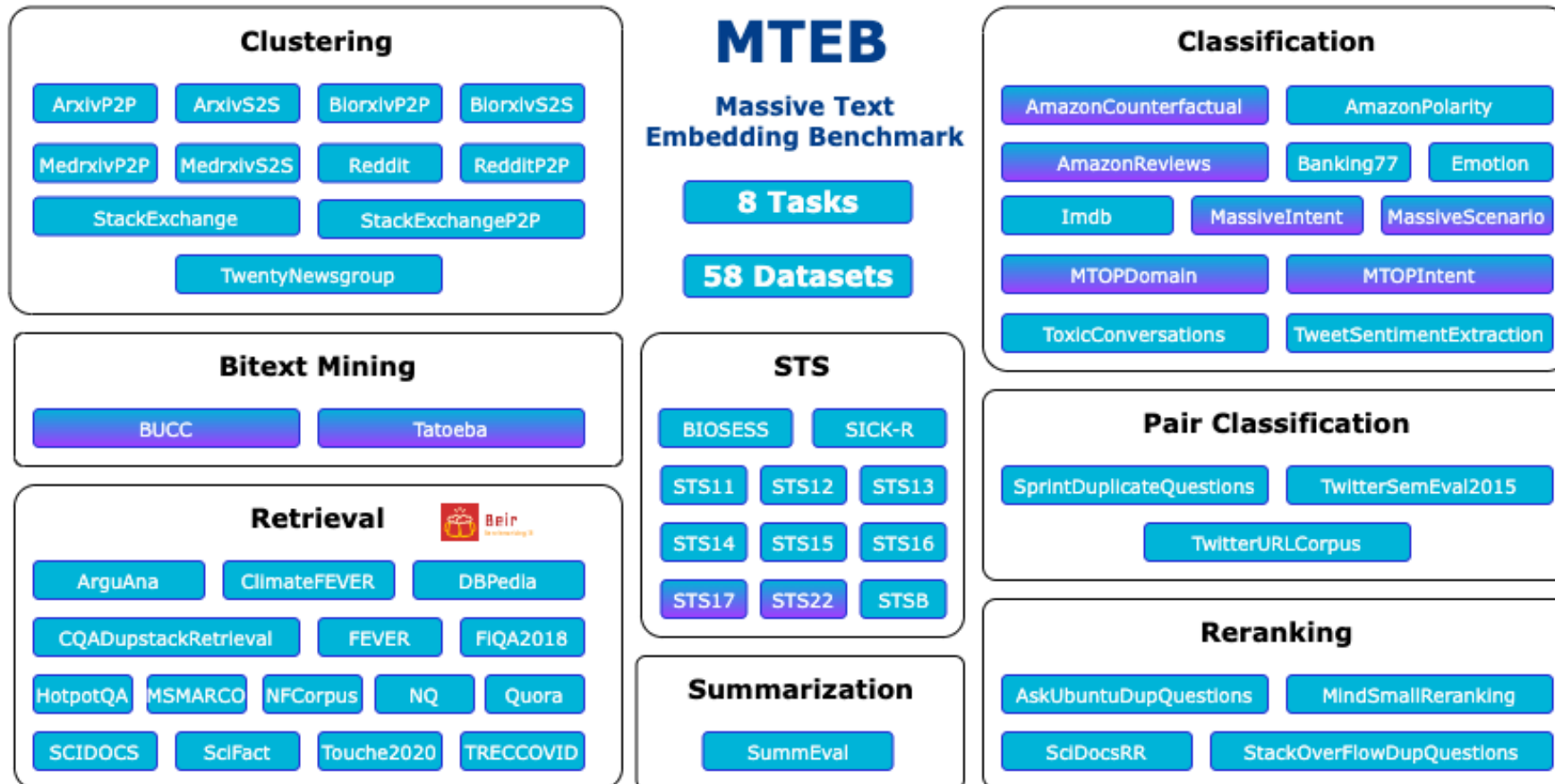
- First, it indexes some documents (e.g. , company HR policies) by embedding them and storing them in a vector index (FAISS).
- At query time, the RetrievalQA chain uses the retriever to find relevant chunks (e.g. , the section of the policy about annual leave) and passes them to the LLM. The LLM then generates an answer like “*Our annual leave policy is ...*” using that retrieved info.

RAG - Embedding Models

- Text embeddings are vector representations of text.
- Dense Embeddings:
 - Capture semantic relationships between words or sentences.
 - Examples: BERT, Sentence-BERT (SBERT), OpenAI's text-embedding models.
 - Effective for understanding context and meaning beyond exact keyword matches.
- Sparse Embeddings:
 - Focus on exact term frequency and presence.
 - Examples: TF-IDF (Term Frequency-Inverse Document Frequency).
 - Useful for precise keyword-based retrieval.

RAG - Embedding Models

- Evaluating the embedding models:
 - MTEB: Massive Text Embedding Benchmark.

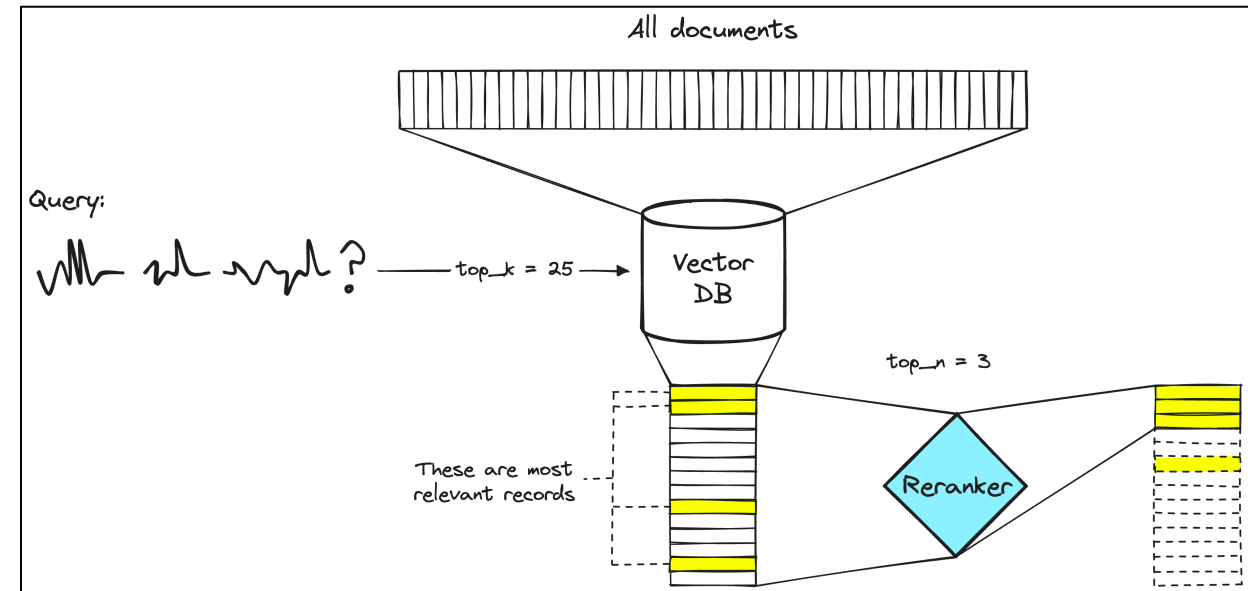


RAG – Text Chunking

- How to break the original corpus into small pieces?
 - *Fixed-size (in characters) overlapping sliding window*: divide text into fixed-size chunks based on character count.
 - *Structure aware splitting (by sentence, paragraph)*: consider the natural structure of text, dividing it based on sentences, paragraphs, sections, or chapters.
 - *Content-aware splitting (markdown, LaTeX, HTML)*: focus on content type and structure, especially in structured documents like Markdown, LaTeX, or HTML. It ensures content types are not mixed within chunks, maintaining integrity.
 - *NLP chunking*: track topic changes based on semantic understanding, dividing text into chunks by detecting significant shifts in topics.

Re-Ranking

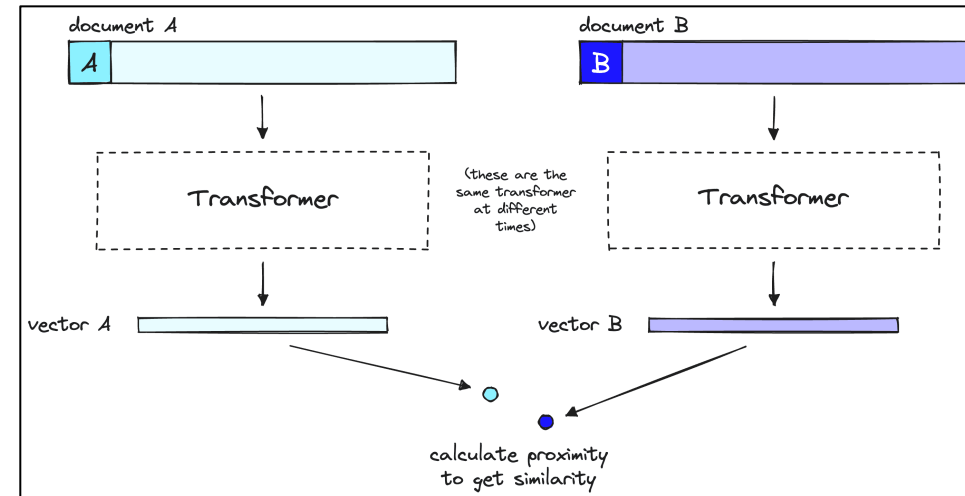
- A reranking model: *given a query and document pair*, will output a similarity score.
- We use this score to reorder the documents by relevance to our query.
- Two stage search:
 - The first-stage model (original embedding model/retriever) retrieves a set of relevant documents from a larger dataset.
 - Then, a second-stage model (the reranker) is used to rerank those documents retrieved by the first-stage model.



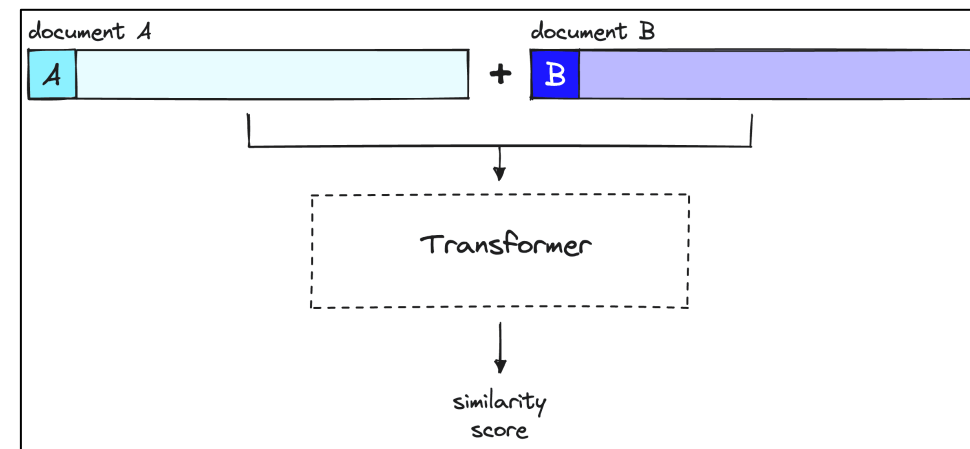
<https://www.pinecone.io/learn/series/rag/rerankers/>

Re-Ranking

- Re-rankers are expected to be much more accurate than embedding models.
- a reranker can receive the raw information directly into the large transformer computation, meaning less information loss.
- Because we are running the reranker at user query time, we have the added benefit of analyzing our document's meaning specific to the user query — rather than trying to produce a generic, averaged meaning.



Embedding

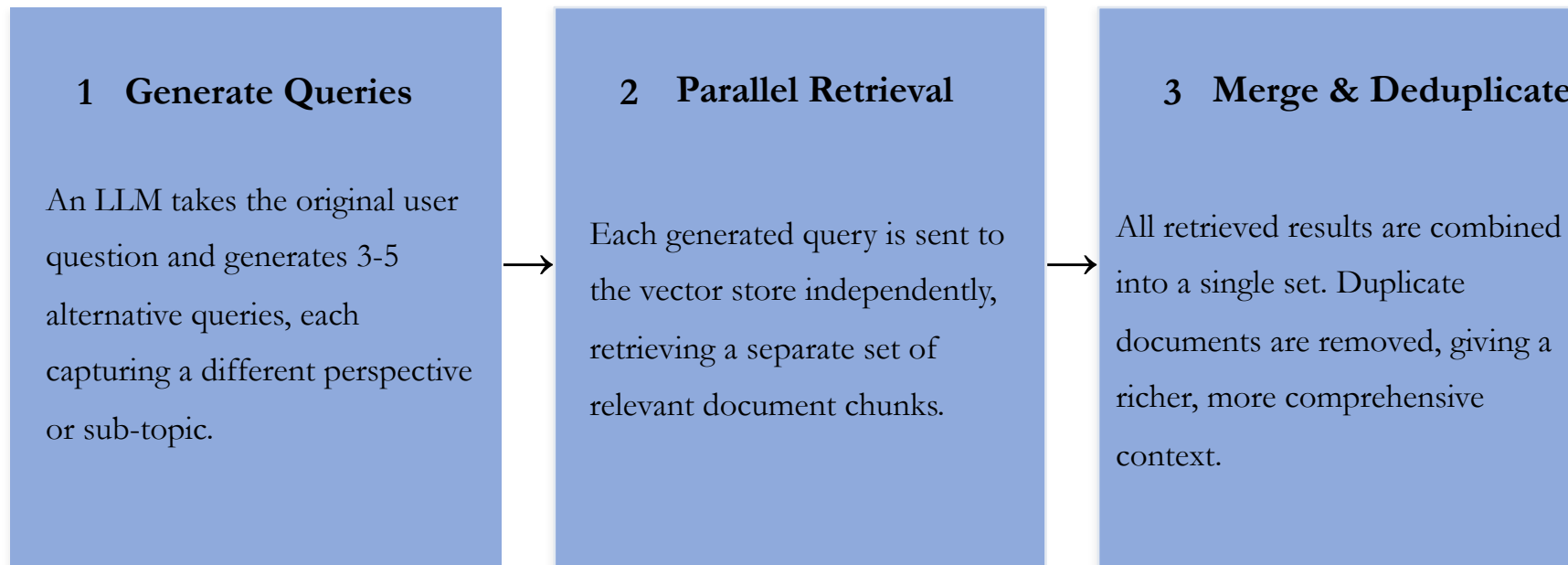


Reranker

Multi Query Retriever

- A single embedding cannot capture multiple facets of a complex question.
- Multi Query Retriever:
- Let LLM generate diverse queries.
- Retrieve in parallel.
- Merge & deduplicate.

"What are the health effects of climate change on coastal populations?"



Example Generated Queries: "climate change health effects" • "coastal flooding public health" • "respiratory illness air quality warming"

Table Augmented Generation



TAG: Table Augmented Generation

- **Goal:** Let users ask arbitrary natural-language questions over data, leveraging both the reasoning of LLMs and the information stored in relational databases.
- Text2SQL converts a natural language query to SQL, but only works for questions expressible in relational algebra.
- RAG augments LLMs with data retrieval, but handles only questions answerable via a few retrieved chunks of text.
- Real user queries often require more: combining up-to-date domain data (i.e., data stored in relational database), world knowledge, and semantic reasoning – e.g., sentiment analysis or trend explanations that pure SQL or simple retrieval can't handle.

Text2SQL is Not Enough: Unifying AI and Databases with TAG

Asim Biswal^{1,7} Liana Patel^{2,*} Siddharth Jha¹ Amog Kamsetty¹ Shu Liu¹
Joseph E. Gonzalez¹ Carlos Guestrin² Matei Zaharia¹
¹UC Berkeley ²Stanford University

ABSTRACT

AI systems that serve natural language questions over databases promise to unlock tremendous value. Such systems would allow users to leverage the powerful reasoning and knowledge capabilities of language models (LMs) alongside the scalable computational power of data management systems. These combined capabilities would empower users to ask arbitrary natural language questions over custom data sources. However, existing methods and benchmarks insufficiently explore this setting. Text2SQL methods focus solely on natural language questions that can be expressed in relational algebra, representing a small subset of the questions real users wish to ask. Likewise, Retrieval-Augmented Generation (RAG) considers the limited subset of queries that can be answered with point lookups to one or a few data records within the database. We propose Table-Augmented Generation (TAG), a unified and general-purpose paradigm for answering natural language questions over databases. The TAG model represents a wide range of interactions between the LM and database that have been previously unexplored and creates exciting research opportunities for leveraging the world knowledge and reasoning capabilities of LMs over data. We systematically develop benchmarks to study the TAG problem and find that standard methods answer no more than 20% of queries correctly, confirming the need for further research in this area. We release code for the benchmark at <https://github.com/TAG-Research/TAG-Bench>.

1 INTRODUCTION

Language models promise to revolutionize data management by letting users ask natural language questions over data, which has led to a great deal of research in Text2SQL and Retrieval-Augmented Generation (RAG) methods. In our experience, however (including from internal workloads and customers at Databricks), users' questions often transcend the capabilities of these paradigms, demanding new research investment towards systems that combine the logical reasoning abilities of database systems with the natural language reasoning abilities of modern language models (LMs).

In particular, we find that real business users' questions often require sophisticated combinations of domain knowledge, world knowledge, exact computation, and semantic reasoning. Database systems clearly provide a source of *domain knowledge* through the up-to-date data they store, as well as *exact computation* at scale (which LMs are bad at).

LMs offer to extend the existing capabilities of databases in two key ways. First, LMs possess *semantic reasoning* capabilities over textual data, a core element of many natural language user queries. For example, a Databricks customer survey showed users wish to ask questions like *which customer reviews of product X are positive?*, or *why did my sales drop during this period?*. These questions present

complex reasoning-based tasks, such as sentiment analysis over free-text fields or summarization of trends. LMs are well-suited to these tasks, which cannot be modeled by the exact computation or relational primitives in traditional database systems.

Secondly, the LM, using knowledge learned during model training and stored implicitly by the model's weights, can powerfully augment the user's data with *world knowledge* that is not captured explicitly by the database's table schema. As an example, a Databricks internal AI user asked *what are the QoQ trends for the 'retail' vertical?* over a table containing attributes for account names, products and revenue. To answer this query the system must understand how the business defines QoQ (e.g., the quarter over quarter trends from the last quarter to the current quarter or this quarter last year to this quarter this year), as well as which companies are considered to be in the *retail vertical*. This task is well-suited to leverage the knowledge held by a pre-trained or fine-tuned LM.

Systems that efficiently leverage databases and LMs together to serve natural language queries, in their full generality, hold potential to transform the way users understand their data. Unfortunately, these questions cannot be answered today by common methods, such as Text2SQL and RAG. While Text2SQL methods [26, 28, 31, 32] are suitable for the subset of natural language queries that have direct relational equivalents, they cannot handle the vast array of user queries that require semantic reasoning or world knowledge. For instance, the previous user query asking *which customer reviews are positive* may require logical row-wise LM reasoning over reviews to classify each as positive or negative. Similarly the question *which asks why sales dropped* entails a reasoning question that must aggregate information across many table entries.

On the other hand, the RAG model is limited to simple relevance-based point lookups to a few data records, followed by a single LM invocation. This model serves only the subset of queries answerable by point lookups and also fails to leverage the richer query execution capabilities of many database systems, which leaves computational tasks (e.g., counting, math and filtering) to a single invocation of the error-prone LM. In addition to being error prone and inefficient at computational tasks, LMs have also been shown to perform poorly on long-context prompts limiting their ability to reason about data at scale in the generation phase of RAG.

We instead propose *table-augmented generation (TAG)* as a unified paradigm for systems that answer natural language questions over databases. Specifically, TAG defines three key steps, as shown in Figure 1. First, the query synthesis step *syn* translates the user's arbitrary natural language request *R* to an executable database query *Q*. Then, the query execution step *exec* executes *Q* on the database system to efficiently compute the relevant data *T*. Lastly, the answer generation step *gen* utilizes *R* and *T*, where the LM is orchestrated, possibly in iterative or recursive patterns over the data, to generate the final natural language answer *A*. The TAG model is simple, but powerful: it is defined by the following three

⁷Both authors contributed equally to this research.

Text2SQL Example



Natural Language Question

"What are the names and email addresses of customers who placed orders in the last 30 days?"



Database Schema

Tables:

- **Customers**
 - `customer_id` (INTEGER)
 - `name` (TEXT)
 - `email` (TEXT)
- **Orders**
 - `order_id` (INTEGER)
 - `customer_id` (INTEGER)
 - `order_date` (DATE)



LLM-Generated SQL Query

sql

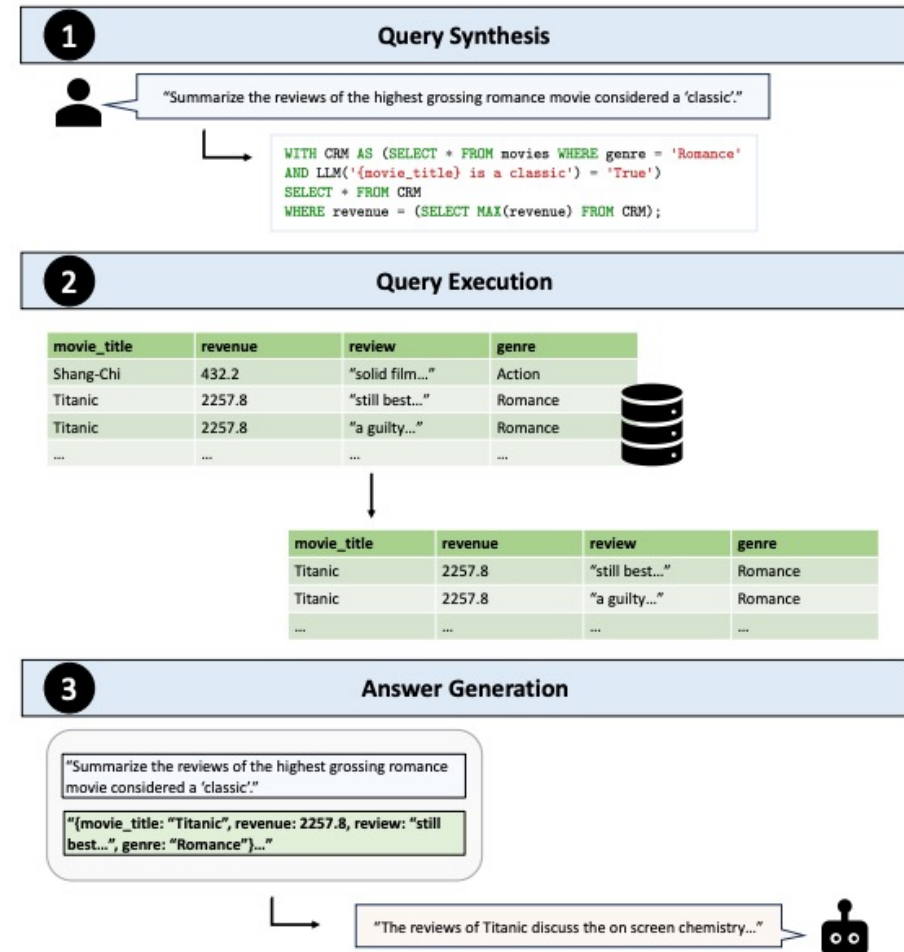
```
SELECT c.name, c.email
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL '30 days';
```

Limitation of Text2SQL

- **Structured queries only**: the question must be fully translated into SQL (i.e., has a direct relational form); it cannot perform tasks beyond SQL's capabilities.
- **No semantic reasoning or external knowledge**: it cannot handle queries requiring understanding free text or using facts not in the database. For example:
 - “Which customer reviews of product X are positive?” – Requires classifying review sentiment (an NL task), which SQL alone cannot do.
 - “Why did my sales drop during this period?” – Involves reasoning across many data points and explaining a trend beyond a single SQL query's scope.
- **Covers a small subset of needs**: In practice, Text2SQL can answer only a limited fraction of realistic data questions.

TAG Pipeline: Three Key Steps

- **Query Synthesis:** The LLM parses the natural language request and generates a database query (e.g. in SQL) to retrieve the needed information.
- **Query Execution:** The generated query is executed on the database, which efficiently processes it over potentially large data.
- **Answer Generation:** Finally, the LLM generates a natural-language answer using the results from the database. It takes the retrieved data (often formatted or summarized into text) and composes the answer to the user's question.



Real-World TAG Scenarios

- Sentiment Analysis & Feedback Mining:
 - TAG enables analytics on textual data stored in tables.
 - For example: “Among the negative reviews of product X, what are the most frequently criticized features?”
 - A TAG system can use the LLM to classify each review as positive/negative during Query Synthesis and filter the table, then aggregate common complaints and have the LLM summarize them.
 - This provides insights that neither SQL nor simple lookup could produce alone.

Real-World TAG Scenarios

- Business Queries with World Knowledge:
 - TAG can handle enterprise questions that involve jargon or background knowledge not explicitly in the database.
 - E.g.: “What are the QoQ trends for the ‘retail’ vertical in our sales data?”
 - The term “retail” might not be a column in the data, but the LLM can infer which entries (companies) are in the retail sector, and the database can compute quarter-over-quarter trends for those, which the LLM then narrates.

Real-World TAG Scenarios

- **Data Summarization at Scale:**
 - TAG shines at questions requiring combining many data points.
 - For instance: “Summarize all races held at Sepang International Circuit.”
 - A TAG system can execute an SQL query to pull all records of races at that circuit and then use the LLM to generate a concise summary of the results (e.g., listing years, winners, and notable facts).
 - Prior approaches would either miss many entries or struggle to compile an answer from dozens of rows.

GraphRAG

GraphRAG Motivation

- Challenges with traditional RAG:
 - Ineffective for "sensemaking" queries like "What are the main themes in the dataset?"
 - Retrieval based on local relevance, missing broader connections.
- Need for structured context:
 - Incorporating relationships between entities can provide deeper insights.
 - Graph structures naturally represent interconnected information.
- Core idea:
 - GraphRAG integrates graph-based representations into the RAG framework.
 - Constructs a knowledge graph from the source corpus to guide retrieval and generation.

arXiv:2404.16130v2 [cs.CL] 19 Feb 2025

From Local to Global: A GraphRAG Approach to Query-Focused Summarization

Darren Edge^{1†} Ha Trinh^{1†} Newman Cheng² Joshua Bradley² Alex Chao³
 Apurva Mody³ Steven Truitt² Dasha Metropolitanaky¹ Robert Osazuwa Ness¹

Jonathan Larson¹

¹Microsoft Research

²Microsoft Strategic Missions and Technologies

³Microsoft Office of the CTO

{daedge, trinhha, newmcheng, joshbradley, achao, moapurva, steventruitt, dasham, robertness, jolarso}@microsoft.com

[†]These authors contributed equally to this work

Abstract

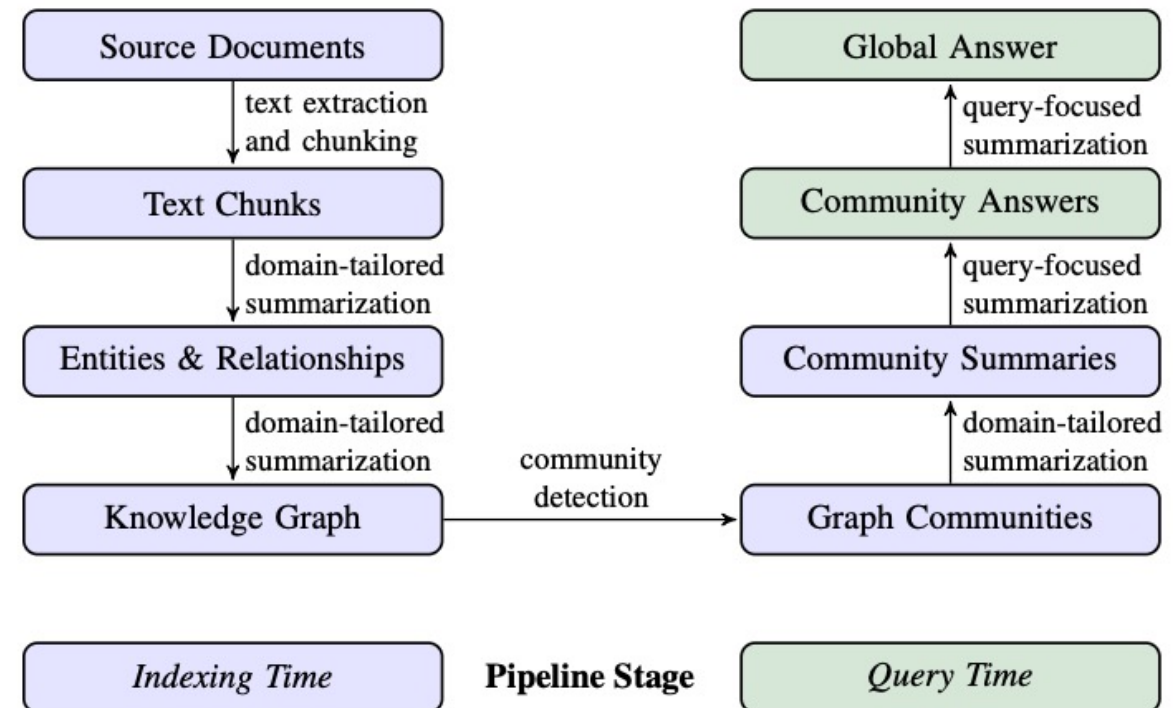
The use of retrieval-augmented generation (RAG) to retrieve relevant information from an external knowledge source enables large language models (LLMs) to answer questions over private and/or previously unseen document collections. However, RAG fails on global questions directed at an entire text corpus, such as "What are the main themes in the dataset?", since this is inherently a query-focused summarization (QFS) task, rather than an explicit retrieval task. Prior QFS methods, meanwhile, do not scale to the quantities of text indexed by typical RAG systems. To combine the strengths of these contrasting methods, we propose *GraphRAG*, a graph-based approach to question answering over private text corpora that scales with both the generality of user questions and the quantity of source text. Our approach uses an LLM to build a graph index in two stages: first, to derive an entity knowledge graph from the source documents, then to pre-generate community summaries for all groups of closely related entities. Given a question, each community summary is used to generate a partial response, before all partial responses are again summarized in a final response to the user. For a class of global sensemaking questions over datasets in the 1 million token range, we show that GraphRAG leads to substantial improvements over a conventional RAG baseline for both the comprehensiveness and diversity of generated answers.

1 Introduction

Retrieval augmented generation (RAG) (Lewis et al., 2020) is an established approach to using LLMs to answer queries based on data that is too large to contain in a language model's *context window*, meaning the maximum number of *tokens* (units of text) that can be processed by the LLM at once (Kuratov et al., 2024; Liu et al., 2023). In the canonical RAG setup, the system has access to a large external corpus of text records and retrieves a subset of records that are individually relevant to the query and collectively small enough to fit into the context window of the LLM. The LLM then

GraphRAG Architecture Overview

- Indexing time:
 - Source documents -> text chunks: the documents in the corpus are split into text chunks.
 - Entity and relation extraction: using LLMs to identify entities and their relationships.
 - Knowledge graph construction: forming a single large knowledge graph from extracted entities and relations.
 - Graph community detection: identifying clusters within the graph.
 - Community summarization: Generating summaries for each community.
- Query time:
 - Query processing: Matching user queries to relevant community summaries.
 - Answer generation: LLM generates responses based on selected summaries.



Case Study - Entity and Relation Extraction

Scenario: Analyzing a corpus of pharmaceutical research papers to understand drug interactions and side effects.

Input corpus: "A recent study indicates that combining Drug A (an ACE inhibitor) with Drug B (a diuretic) significantly reduces blood pressure in patients with hypertension. However, this combination may increase the risk of renal impairment."

- **Extracted Entities:**

- Drug A
- ACE inhibitor
- Drug B
- Diuretic
- Blood pressure
- Hypertension
- Medium
- Renal impairment

- **Extracted Relations:**

- (Drug A, *is a type of*, ACE inhibitor)
- (Drug B, *is a type of*, Diuretic)
- (Drug A + Drug B, *reduces*, Blood pressure)
- (Drug A + Drug B, *treats*, Hypertension)
- (Drug A + Drug B, *may cause*, Renal impairment)

Case Study - Knowledge Graph Construction

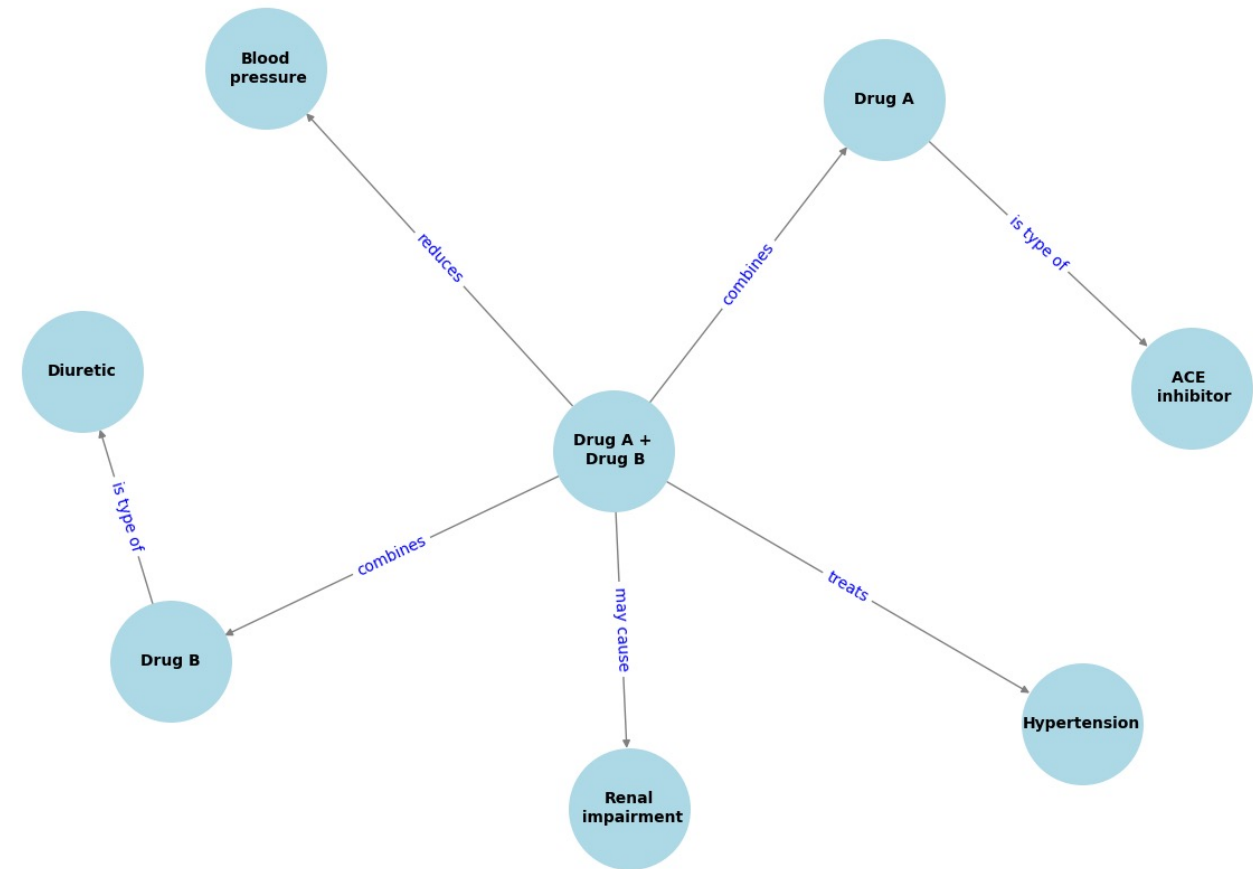
- Extracted Entities:

- Drug A
- ACE inhibitor
- Drug B
- Diuretic
- Blood pressure
- Hypertension
- Medium
- Renal impairment

- Extracted Relations:

- (Drug A, *is a type of*, ACE inhibitor)
- (Drug B, *is a type of*, Diuretic)
- (Drug A + Drug B, *reduces*, Blood pressure)
- (Drug A + Drug B, *treats*, Hypertension)
- (Drug A + Drug B, *may cause*, Renal impairment)

- The combination of Drug A and Drug B is represented as a composite node to capture their combined effects.



Case Study - Community Detection and Summarization

Community Detection:

- **Community 1: Antihypertensive Agents**
 - Drug A
 - ACE inhibitor
 - Drug B
 - Diuretic
 - Blood pressure
 - Hypertension
- **Community 2: Adverse Effects**
 - Drug A + Drug B
 - Renal impairment

Community Summaries:

- **Antihypertensive Agents:**
 - "Drug A, an ACE inhibitor, and Drug B, a diuretic, are commonly used in combination to effectively reduce blood pressure in hypertensive patients."
- **Adverse Effects:**
 - "While the combination of Drug A and Drug B is effective in managing hypertension, it may increase the risk of renal impairment."

Case Study - Query Processing and Answer Generation

User Query: "What are the benefits and risks of combining ACE inhibitors with diuretics in hypertension treatment?"

Processing Steps:

- **Query Understanding:** Identify key entities: ACE inhibitors, diuretics, hypertension, benefits, risks.
- **Community Matching:** Match query to relevant communities: Antihypertensive Agents and Adverse Effects.
- **Information Retrieval:** Retrieve summaries and relations from matched communities.

Answer Generation: "Combining ACE inhibitors with diuretics is effective in lowering blood pressure among hypertensive patients. However, this combination may increase the risk of renal impairment, necessitating careful monitoring."

Comparison with Standard RAG

- Standard RAG:
 - Relies on vector similarity for retrieval.
 - Limited in capturing structural relationships.
- GraphRAG:
 - Utilizes graph structures to represent knowledge.
 - Enhances retrieval by considering entity relationships and community contexts.
- Example:
 - Query: "What are the main themes in the dataset?"
 - Standard RAG Response: May retrieve unrelated documents based on keyword similarity.
 - GraphRAG Response: Provides a summary of interconnected themes identified through community detection.

Summary & References

- Standard RAG can be used to augment the generation quality by including local text data in the prompt.
 - TAG can interact with relational data, which most current information systems are built on top of.
 - GraphRAG organizes the raw text data in a more complicated structure to further improve the generation quality.
-
- <https://www.together.ai/blog/rag-tutorial-langchain>
 - <https://huggingface.co/blog/mteb>
 - <https://arxiv.org/pdf/2408.14717>
 - <https://arxiv.org/abs/2404.16130>